

RK3562_Linux 应用开发说明



主题	RK3562_Linux 应用开发说明
文档号	1.0
创建时间	2025-04-18
最后修改	2025-04-18
版本号	1.0
文件名	RK3562_Linux 应用开发说明 V1.0.pdf
文件格式	Portable Document Format

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2025-04-18	米泓宇	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
前言	6
第 1 章 Linux 常用开发案例	7
1.1 ADC 案例 (adc_demo)	7
1.1.1 案例功能	7
1.1.2 操作说明	7
1.1.3 运行结果	8
1.1.4 部分代码	9
1.2 GPIO 案例 (gpio_demo)	11
1.2.1 案例功能	11
1.2.2 操作说明	11
1.2.3 运行结果	13
1.2.4 部分代码	14
1.3 PWM 案例 (pwm_demo)	16
1.3.1 案例功能	16
1.3.2 操作说明	16
1.3.3 运行结果	17
1.3.4 部分代码	18

1.4 UART 案例 (uart_demo)	20
1.4.1 案例功能	20
1.4.2 操作说明	20
1.4.3 运行结果	21
1.4.4 部分代码	22
1.5 CAN 案例 (can_demo)	24
1.5.1 案例功能	24
1.5.2 操作说明	24
1.5.3 运行结果	25
1.5.4 部分代码	26

前言

本文档涉及的开发案例位于产品资料“5、外设例程\”下。需要特别说明的是：

(1) 通用性说明

- ◆ 本文档中的案例适用于 **RK3562 核心板** 平台的所有型号开发板。
- ◆ 不同型号的开发板外设配置（外设的可用性以及引脚的定义）存在差异，阅读本文档涉及的案例时，请查阅对应开发板型号《测试手册》中的外设测试部分。

(2) 案例目录结构

- ◆ 案例采用标准工程结构，包含以下内容：

```
.
├── xxx_demo
│   ├── inc      # 案例头文件目录 (.h)
│   ├── src      # 案例源文件目录 (.c)
│   ├── xxx_demo # 预编译可执行文件（兼容我司标准文件系统）
│   └── Makefile # Makefile 文件
```

(3) 使用说明

- ◆ 直接运行：可直接执行编译好的 **xxx_demo** 文件
- ◆ 重新编译：
 - 1) 参考对应开发板《开发环境搭建》文档正确配置开发环境。
 - 2) 将案例目录整体拷贝至开发环境。
 - 3) 修改 **Makefile** 文件中的 **gcc** 编译链为开发环境中配置的交叉编译链。
 - 4) 执行编译命令。

```
cd xxx_demo/

make
```

- 5) 编译完成后会在当前目录生成新的可执行文件。

注：本文档因篇幅限制仅展示部分代码片段，完整代码请从配套资料包中获取。

第 1 章 Linux 常用开发案例

1.1 ADC 案例 (adc_demo)

1.1.1 案例功能

ADC (Analog-to-Digital Converter) 可以将模拟信号转换为数字信号, ADC 驱动使用 Linux 标准 I/O 子系统 ADC 设备接口, 本案例功能为读取指定 ADC 通道的原始值, 计算为电压值并打印。

本案例适用的开发板型号:

- HZ-RK3562_MiniEVM
- HZ-RK3562_EVM

1.1.2 操作说明

(1) 赋予可执行权限

将本案例根目录下的可执行文件 `adc_demo` 拷贝至开发板文件系统中, 需要确保文件具有可执行权限:

```
chmod +x adc_demo
```

```
./adc_demo
```

(2) 案例详情

直接运行案例, 不带任何参数, 将会输出案例的使用介绍与详情, 如图 1.1-1。

```
使用方法：
./adc_demo [options] <device_num> <channel_name> [monitor] [interval_ms] [duration_sec]
单次读取：./adc_demo <device_num> <channel_name>
实时监控：./adc_demo <device_num> <channel_name> monitor [interval_ms] [duration_sec]
显示帮助：./adc_demo -h 或 --help
参数说明：
device_num:      ADC设备号 (0-1), 对应 SARADC0-SARADC1
channel_name:    通道名称, 如 in_voltage3_raw
monitor:         实时监控模式
interval_ms:     监控间隔, 单位毫秒, 默认为 500ms
duration_sec:    监控时长, 单位秒, 默认为 0 (无限监控直到Ctrl+C)
选项：
-h, --help:      检测系统中可用的ADC通道列表
-v, --verbose:   显示详细信息和计算流程 (单次读取时)

使用 -h 或 --help 参数可查看可用的ADC通道
```

图 1.1-1 adc_demo 详情

(3) 使用示例

```
./adc_demo 0 in_voltage3_raw monitor
```

表示持续监控 SARADC0_IN3, 即 pin5 处的电压。监控参数使用默认参数, 即 500 毫秒间隔, 无限监控直到手动停止。

1.1.3 运行结果

使用 1.1.2 (3) 中的示例:

可参考对应板卡型号的《测试手册》中 ADC 功能测试小节, 根据实际需求选择目标测试引脚。

◆ 程序使用实时监控模式运行时, 选定引脚接入 1.8V 电压, 由图 1.1-2 运行结果可见, 程序通过该引脚采集到 1.8V 电压。


```

开始监控 ADC设备 ...
开始监控 ADC设备 0, 通道 in_voltage3_raw
实时数据显示 ( 每 500 毫秒采样一次, 按 Ctrl+C 停止 ) :
时间 (秒)  原始值  电压值 (V)
-----
0.00      489      0.860
0.50      489      0.860
1.00      490      0.861
1.50      492      0.865
2.00      491      0.863
2.50      490      0.861
3.00     1023      1.798
3.50     1023      1.798
4.00     1023      1.798
4.50     1023      1.798
5.00     1023      1.798
5.50     1023      1.798
6.00     1023      1.798
6.50      500      0.879
^C
接收到终止信号, 正在退出 ...
-----
监控结束, 共获取 14 个样本

```

图 1.1-2 adc_demo 运行结果

1.1.4 部分代码

- (1) ReadAdcRawValue - 通过 sysfs 文件系统访问 Linux 系统下的 ADC 设备, 读取 ADC 原始值。

```

92 // 读取ADC原始值
93 int ReadAdcRawValue(const char *devicePath, const char *channelName, int *rawValue)
94 {
95     if (!devicePath || !channelName || !rawValue) {
96         return -1;
97     }
98
99     char path[MAX_PATH_LEN];
100     snprintf(path, sizeof(path), "%s/%s", devicePath, channelName);
101
102     int fd = open(path, O_RDONLY);
103     if (fd == -1) {
104         fprintf(stderr, "无法打开ADC通道 %s: %s\n", channelName, strerror(errno));
105         return -1;
106     }
107
108     char buf[32];
109     ssize_t len = read(fd, buf, sizeof(buf) - 1);
110     close(fd);
111
112     if (len <= 0) {
113         fprintf(stderr, "读取ADC值失败: %s\n", strerror(errno));
114         return -1;
115     }
116
117     buf[len] = '\0';
118     *rawValue = atoi(buf);
119     return 0;
120 }

```

图 1.1-3 adc_demo 读取 ADC 原始值函数

(2) ReadAdcVoltage-将原始值通过计算转化为电压值:

ADC 电压计算公式:

$$V_{in} = V_{raw} \times [V_{ref} \div (2^n - 1)]$$

例如, 若 ADC 采样值为 847, 实际输入电压值 $V_{in} = 847 \times [1.8 \div (2^{10} - 1)] \approx 1.49 \text{ V}$, 结果与输入电压 1.5V 相近。

参数解析:

V_{in} : 实际输入电压值;

V_{raw} : ADC 采样值;

V_{ref} : 参考电压值, V_{ref} 为 1.8V;

n : ADC 转换精度, 当前 ADC 转换精度为 10 位, 因此 $n=10$ 。

```
122 // 读取ADC并计算电压值
123 int ReadAdcVoltage(int deviceNum, const char *channelName, AdcReadResult *result)
124 {
125     if (!channelName || !result) {
126         return -1;
127     }
128
129     char devicePath[MAX_PATH_LEN];
130     BuildAdcDevicePath(devicePath, sizeof(devicePath), deviceNum);
131
132     if (ReadAdcRawValue(devicePath, channelName, &result->rawValue) != 0) {
133         return -1;
134     }
135
136     // 计算电压值:  $V_{in} = V_{raw} * [V_{ref} / (2^n - 1)]$ 
137     result->voltage = (float)result->rawValue * ADC_REF_VOLTAGE / ADC_RESOLUTION;
138
139     return 0;
140 }
```

图 1.1-4 adc_demo 计算电压值函数

1.2 GPIO 案例 (gpio_demo)

1.2.1 案例功能

GPIO (General-Purpose Input/Output) 是一种通用输入输出接口，可灵活配置为输入或输出模式，并控制其电平状态。部分 GPIO 支持中断功能，同时还具备复用能力，可配置为 I2C、PWM 等特殊功能接口。本案例提供便捷的 GPIO 配置功能，包括方向设置（输入/输出）、输出电平控制等操作。

本案例适用的开发板型号：

■ HZ-RK3562_MiniEVM

1.2.2 操作说明

(1) 赋予可执行权限

将本案例根目录下的可执行文件 `gpio_demo` 拷贝至开发板文件系统中，需要确保文件具有可执行权限，且运行需要以 root 用户身份（`sudo`）运行：

```
chmod +x gpio_demo
```

```
sudo ./gpio_demo
```

(2) 案例详情

直接运行案例，不带任何参数，将会输出案例的使用介绍与详情，如图 1.2-1。

```
Usage:
./gpio_demo <gpio_num> <mode> [value]
使用方法：
输出模式：./GpioDemo <gpio_num> out <high|low>
输入模式：./GpioDemo <gpio_num> in
中断模式：./GpioDemo <gpio_num> int <rising|falling|both>
释放模式：./GpioDemo <gpio_num> release
运行程序需要 sudo 权限
参数说明：
gpio_num: GPIO 编号
mode:      工作模式
    out    - 输出模式，需要指定输出电平 high 或 low，设置后自动退出
    in     - 输入模式，持续监控读取当前电平状态，按 Ctrl+C 退出
    int    - 中断模式，持续监控指定触发方式的中断，按 Ctrl+C 退出
    release - 释放模式，释放指定的 GPIO 资源
value:     根据模式指定的参数
    输出模式：high(高电平)，low(低电平)
    中断模式：rising(上升沿)，falling(下降沿)，both(双边沿)
```

图 1.2-1 gpio_demo 详情

(3) 使用示例

```
sudo ./gpio_demo 19 out high
```

即表示配置 19 号 GPIO 方向为输出，且输出高电平。

```
sudo ./gpio_demo 20 in
```

即表示配置 20 号 GPIO 方向为输入，持续检测输入电平为高或者为低。

1.2.3 运行结果

使用 1.2.2 (3) 中的示例：

可参考对应板卡型号的《测试手册》中 GPIO（开入/开出）功能测试小节，根据实际需求选择目标测试引脚。

- 当配置 GPIO19 方向为输出，电平为高时，此时在对应引脚接上 LED 灯，灯会点亮；接上万用表将会有读数。

- 当配置 GPIO20 方向为输入时，对应引脚接入高低电平，程序会显示此引脚状态，如图 1.2-2。

```
配置 GPIO 19 为输出模式，设置电平为 高 ...
GPIO 19 已设置为输出模式，当前电平为 高
GPIO设置完成，电平将保持
root@rk3562-buildroot:~#
root@rk3562-buildroot:~# ./gpio_demo 20 in
配置 GPIO 20 为输入模式 ...
GPIO 20 已设置为输入模式，当前电平为 低
持续监控 GPIO输入状态，按 Ctrl+C退出 ...
GPIO 20 电平变化为：高
GPIO 20 电平变化为：低
GPIO 20 电平变化为：高
GPIO 20 电平变化为：低
GPIO 20 电平变化为：高
GPIO 20 电平变化为：低
^C
接收到终止信号，正在退出 ...
程序已退出，GPIO 20 资源已释放
```

图 1.2-2 gpio_demo 运行结果

1.2.4 部分代码

- 1) 这段代码展示了如何通过 sysfs 文件系统控制 Linux 系统下的 GPIO 设备。

```
34 // 导出GPIO
35 int GpioExport(int gpioNum)
36 {
37     char path[MAX_PATH_LEN];
38     int fd;
39     int len;
40
41     snprintf(path, sizeof(path), "%s/export", SYSFS_GPIO_PATH);
42     fd = open(path, O_WRONLY);
43     if (fd < 0) ...
44
45     char buf[20];
46
47     len = snprintf(buf, sizeof(buf), "%d", gpioNum);
48     if (write(fd, buf, len) != len) ...
49
50     close(fd);
51
52     // 等待GPIO节点创建完成
53     usleep(100000); // 等待100ms
54     return 0;
55 }
56
57 // 取消导出GPIO
58 int GpioUnexport(int gpioNum)
59 {
60     char path[MAX_PATH_LEN];
61     int fd;
62     int len;
63
64     snprintf(path, sizeof(path), "%s/unexport", SYSFS_GPIO_PATH);
65     fd = open(path, O_WRONLY);
66     if (fd < 0) ...
67
68     char buf[20];
69
70     len = snprintf(buf, sizeof(buf), "%d", gpioNum);
71     if (write(fd, buf, len) != len) ...
72
73     close(fd);
74     return 0;
75 }
```

图 1.2-3 gpio_demo 控制 GPIO 设备

- 2) 这两个函数展示了如何设置 GPIO 的方向（输入或输出）以及如何设置输出值（高电平或低电平）。

```

98 // 设置GPIO方向
99 int GpioSetDirection(int gpioNum, const char *direction)
100 {
101     char path[MAX_PATH_LEN];
102     int fd;
103     int len;
104
105     snprintf(path, sizeof(path), "%s/gpio%d/direction", SYSFS_GPIO_PATH, gpioNum);
106     fd = open(path, O_WRONLY);
107     if (fd < 0) ...
108
109     len = write(fd, direction, strlen(direction));
110     if (len < 0) ...
111
112     close(fd);
113     return 0;
114 }
115
116 // 设置GPIO输出值
117 int GpioSetValue(int gpioNum, int value)
118 {
119     char path[MAX_PATH_LEN];
120     int fd;
121     int len;
122
123     snprintf(path, sizeof(path), "%s/gpio%d/value", SYSFS_GPIO_PATH, gpioNum);
124     fd = open(path, O_WRONLY);
125     if (fd < 0) ...
126
127     char buf = value ? '1' : '0';
128     len = write(fd, &buf, 1);
129     if (len < 0) ...
130
131     close(fd);
132     return 0;
133 }

```

图 1.2-4 gpio_demo 配置 GPIO 参数

1.3 PWM 案例 (pwm_demo)

1.3.1 案例功能

本案例通过配置 PWM（脉宽调制）来控制 LED（D42）的闪烁模式。其中可以通过配置周期来控制 LED 的闪烁频率；通过配置占空比来控制一个周期内，LED 灯点亮和熄灭的时间比例。

本案例适用的开发板型号：

■ HZ-RK3562_MiniEVM

1.3.2 操作说明

(1) 赋予可执行权限

将本案例根目录下的可执行文件 `pwm_demo` 拷贝至开发板文件系统中，需要确保文件具有可执行权限，且运行需要以 root 用户身份（`sudo`）运行：

```
chmod +x pwm_demo
```

```
sudo ./pwm_demo
```

(2) 案例详情

直接运行案例，不带任何参数，将会输出案例的使用介绍与详情，如图 1.3-1。

```
PWM控制程序 - 用于设置和控制PWM输出
使用方法：
./pwm_demo <pwm_chip> <pwm_num> <period_s> <duty_percent> - 配置并启动 PWM
./pwm_demo clean <pwm_chip> <pwm_num> - 停止并清理 PWM
./pwm_demo list - 列出所有 PWM设备

参数说明：
pwm_chip - PWM芯片编号，如 1代表 pwmchip1
pwm_num - PWM通道编号，如 0代表 pwm0
period_s - 周期时间，单位为秒，如 1.0代表 1秒
duty_percent - 占空比，单位为百分比 (0-100)，如 50代表 50%

注意：运行程序需要 sudo 权限
示例：sudo ./pwm_demo 1 0 1.0 50 - 设置 pwmchip1/pwm0 的周期为 1秒，占空比 50%
```

图 1.3-1 pwm_demo 详情

(3) 使用示例

```
sudo ./pwm_demo 1 0 5 90
```


即表示 LED 灯以 5 秒的周期闪烁，这 5 秒内，灯熄灭的时间占 90%即 4.5 秒，灯点亮的时间占 10%，即 0.5 秒。

1.3.3 运行结果

使用 1.3.2 (3) 中的示例：

图 1.3-2 中所示的 LED 灯将会以配置的 PWM 参数进行闪烁。

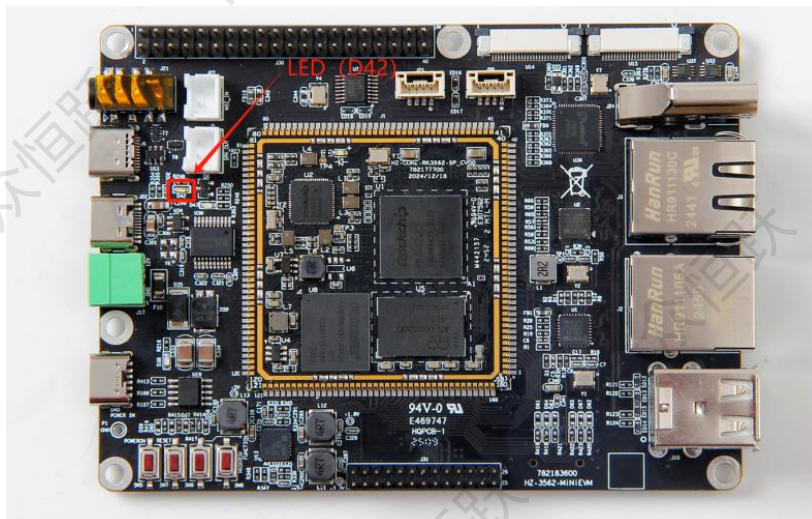


图 1.3-2 pwm_demo 控制的 LED 灯

```
正在配置 PWM 设备：pwmchip1/pwm0
  周期：5.000000 秒 (500000000 纳秒)
  占空比：90.00% (450000000 纳秒)
PWM 设备已导出：pwmchip1/pwm0
当前 PWM 状态：已启用
PWM 禁用成功：pwmchip1/pwm0
正在初始化 PWM 参数...
当前 PWM 周期：100000000 ns
警告：当前占空比 (200000000 ns) 大于要设置的周期 (1000 ns)，先设置占空比为 0
PWM 周期设置成功：1000 ns
当前 PWM 周期：1000 ns
PWM 占空比设置成功：0 ns
当前 PWM 周期：1000 ns
PWM 周期设置成功：500000000 ns
当前 PWM 周期：500000000 ns
PWM 占空比设置成功：450000000 ns
当前 PWM 状态：已禁用
PWM 启用成功：pwmchip1/pwm0
PWM 设备已成功配置并启动
操作成功完成。如需停止 PWM，请运行：./pwm_demo clean 1 0
```

图 1.3-3 pwm_demo 运行结果

1.3.4 部分代码

1) 这段代码展示了如何通过 Linux sysfs 文件系统访问 PWM 设备，实现了设备的导出操作，使得用户空间程序能够控制硬件 PWM 输出。

```
102 int PwmExport(int pwmChip, int pwmNum)
103 {
104     char path[BUF_SIZE];
105
106     /* 检查PWM设备是否已经导出 */
107     if (SafePathJoin(path, BUF_SIZE, PWM_PATH, "") < 0) {
108         return -1;
109     }
110
111     snprintf(path, BUF_SIZE, "%s/pwmchip%d/pwm%d", PWM_PATH, pwmChip, pwmNum);
112     if (access(path, F_OK) == 0) {
113         printf("PWM设备已导出: pwmchip%d/pwm%d\n", pwmChip, pwmNum);
114         return 0; /* 已经导出, 视为成功 */
115     }
116
117     /* 导出PWM设备 */
118     snprintf(path, BUF_SIZE, "%s/pwmchip%d/export", PWM_PATH, pwmChip);
119     if (WriteIntToFile(path, pwmNum) < 0) {
120         fprintf(stderr, "导出PWM设备失败: pwmchip%d/pwm%d\n", pwmChip, pwmNum);
121         return -1;
122     }
123
124     printf("PWM设备导出成功: pwmchip%d/pwm%d\n", pwmChip, pwmNum);
125
126     /* 等待设备节点创建完成 */
127     usleep(200000); /* 200ms - 增加延迟时间确保文件系统操作完成 */
128
129     /* 再次确认PWM设备是否已经导出 */
130     snprintf(path, BUF_SIZE, "%s/pwmchip%d/pwm%d", PWM_PATH, pwmChip, pwmNum);
131     if (access(path, F_OK) != 0) {
132         fprintf(stderr, "导出PWM设备后节点未创建: pwmchip%d/pwm%d\n", pwmChip, pwmNum);
133         return -1;
134     }
135
136     return 0;
137 }
```

图 1.3-4 pwm_demo 访问 PWM 设备

2) 这两个函数展示了如何设置 PWM 的关键参数：周期和占空比。特别是占空比设置函数还实现了参数合法性检查，确保占空比不会超过周期值。

```
154 int PwmSetPeriod(int pwmChip, int pwmNum, unsigned long periodNs)
155 {
156     char path[BUF_SIZE];
157     char buf[BUF_SIZE];
158
159     /* 设置PWM周期 */
160     snprintf(path, BUF_SIZE, "%s/pwmchip%d/pwm%d/period", PWM_PATH, pwmChip, pwmNum);
161
162     /* 先读取当前周期值 */
163     if (ReadFileContent(path, buf, BUF_SIZE) == 0) { ...
164
165     /* 写入新周期值 */
166     if (WriteULongToFile(path, periodNs) < 0) { ...
167
168     printf("PWM周期设置成功: %lu ns\n", periodNs);
169     return 0;
170 }
171
172 int PwmSetDutyCycle(int pwmChip, int pwmNum, unsigned long dutyCycleNs)
173 {
174     char path[BUF_SIZE];
175     char periodPath[BUF_SIZE];
176     char buf[BUF_SIZE];
177     unsigned long currentPeriod = 0;
178
179     /* 读取当前周期值 */
180     snprintf(periodPath, BUF_SIZE, "%s/pwmchip%d/pwm%d/period", PWM_PATH, pwmChip, pwmNum);
181     if (ReadFileContent(periodPath, buf, BUF_SIZE) == 0) {
182         currentPeriod = strtoul(buf, NULL, 10);
183         printf("当前PWM周期: %lu ns\n", currentPeriod);
184
185         /* 确保占空比不超过周期 */
186         if (dutyCycleNs > currentPeriod) {
187             fprintf(stderr, "警告: 占空比(%lu ns)大于周期(%lu ns), 自动调整\n",
188                     dutyCycleNs, currentPeriod);
189             dutyCycleNs = currentPeriod;
190         }
191
192     /* 设置PWM占空比 */
193     snprintf(path, BUF_SIZE, "%s/pwmchip%d/pwm%d/duty_cycle", PWM_PATH, pwmChip, pwmNum);
194     if (WriteULongToFile(path, dutyCycleNs) < 0) {
195         fprintf(stderr, "设置PWM占空比失败: pwmchip%d/pwm%d\n", pwmChip, pwmNum);
196         return -1;
197     }
198
199     printf("PWM占空比设置成功: %lu ns\n", dutyCycleNs);
200     return 0;
201 }
```

图 1.3-5 pwm_demo 配置 PWM 关键参数

1.4 UART 案例 (uart_demo)

1.4.1 案例功能

UART (Universal Asynchronous Receiver/Transmitter) 是一种通用异步收发传输接口, 支持全双工串行通信。本案例通过配置指定串口设备, 实现数据的发送与接收功能。

本案例适用的开发板型号:

- HZ-RK3562-SP_EVM
- HZ-RK3562_EVM

1.4.2 操作说明

(4) 赋予可执行权限

将本案例根目录下的可执行文件 `uart_demo` 拷贝至开发板文件系统中, 需要确保文件具有可执行权限, 且运行需要以 `root` 用户身份或使用 `sudo` 运行:

```
chmod +x uart_demo
```

```
sudo ./uart_demo
```

(5) 案例详情

直接运行案例, 不带任何参数, 将会输出案例的使用介绍与详情, 如图 1.4-1。

```
Usage:
./uart_demo <设备路径> -w <数据> -b <波特率>    (发送模式)
./uart_demo <设备路径> -r -b <波特率>          (接收模式)
使用方法:
发送数据: ./uart_demo <设备路径> -w <数据> -b <波特率>
接收数据: ./uart_demo <设备路径> -r -b <波特率>
运行程序需要 sudo 权限
参数说明:
设备路径: 串口设备文件, 例如 /dev/ttyS0, /dev/ttyUSB0 等
-w: 发送模式, 后接要发送的数据
-r: 接收模式
-b: 设置波特率, 如果不指定, 默认为 115200
-h: 显示帮助信息
```

图 1.4-1 uart_demo 详情

(6) 使用示例

```
./uart_demo /dev/ttyAS1 -w hello
```

表示通过 `ttyAS1` 这个串口设备发送数据，波特率为默认 115200。

```
./uart_demo /dev/ttyAS1 -r
```

表示通过 `ttyAS1` 这个串口设备接收数据，波特率为默认 115200。

1.4.3 运行结果

使用 1.4.2 (3) 中的示例：

可参考对应板卡型号的《测试手册》中 UART（RS485/RS232）功能测试小节，根据实际需求选择目标测试引脚。

运行结果如下所示：

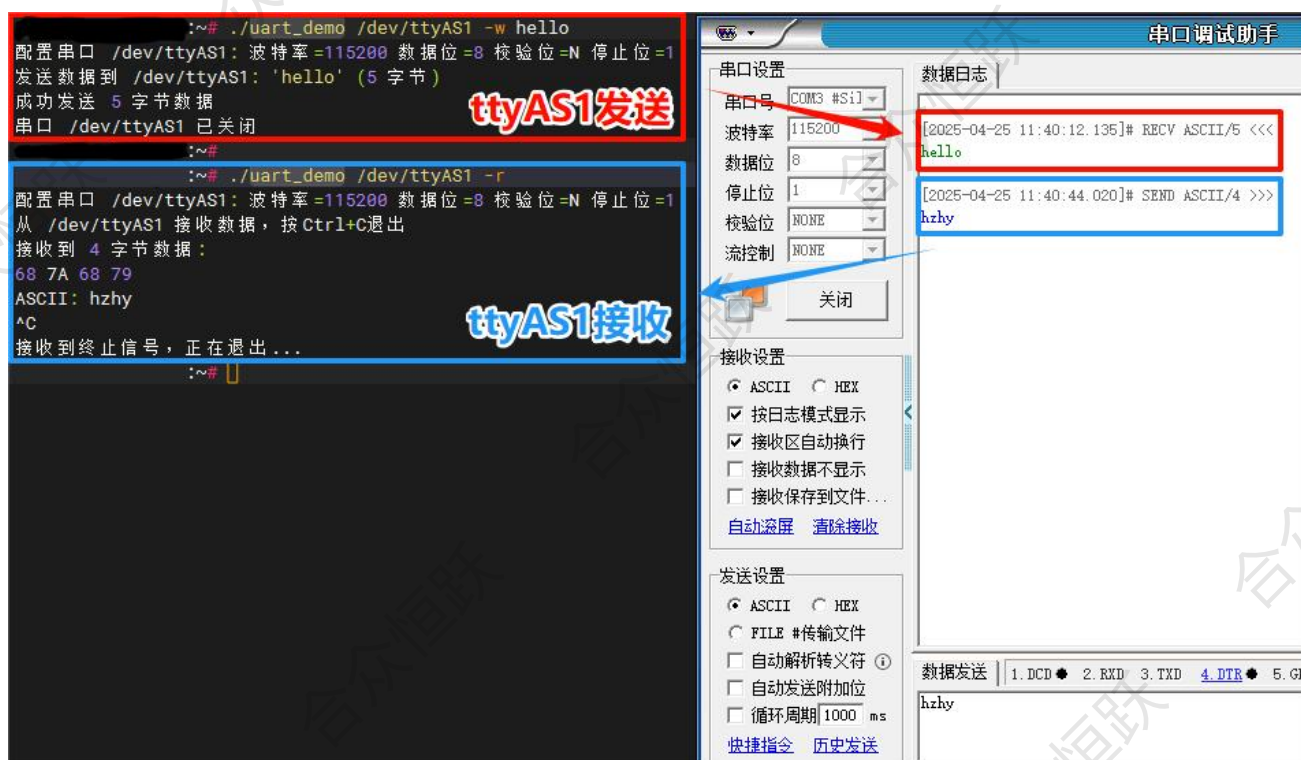


图 1.4-2 uart_demo 运行结果

1.4.4 部分代码

- 1) UartSendAndPrint- 通过调用 UartSend 函数，发送数据并打印发送的数据内容。

```
// 发送数据并打印结果
int UartSendAndPrint(int fd, const char *device, const char *data)
{
    int len, ret;

    if (fd < 0 || data == NULL || device == NULL) {
        return -1;
    }

    len = strlen(data);

    printf("发送数据到 %s: '%s' (%d 字节)\n", device, data, len);

    ret = UartSend(fd, data, len);
    if (ret < 0)
    {
        printf("发送数据失败\n");
        return -1;
    }

    printf("成功发送 %d 字节数据\n", ret);
    return ret;
}
```

图 1.4-3 uart_demo 发送数据

- 2) uartReceiveAndPrint - 通过调用 UartReceive 函数，接收数据并打印。

```
// 接收串口数据并打印（带超时检查和退出标志）
int UartReceiveAndPrint(int fd, int *running_flag)
{
    unsigned char buffer[MAX_BUFFER_SIZE];
    int ret;

    // 检查程序是否被要求退出
    if (!running_flag || !(*running_flag) || fd < 0) {
        return -1;
    }

    // 使用固定的短超时时间，以便能够定期检查running_flag
    ret = UartReceive(fd, buffer, sizeof(buffer) - 1, 500); // 500毫秒超时
    if (ret <= 0) {
        return ret; // 超时或出错，返回但不中断主循环
    }

    buffer[ret] = 0; // 确保结束符

    printf("接收到 %d 字节数据:\n", ret);
    UartPrintHexData(buffer, ret);

    return ret;
}
```

图 1.4-4 uart_demo 接收数据

1.5 CAN 案例 (can_demo)

1.5.1 案例功能

CAN (Controller Area Network) 是 ISO 国际标准化的一种串行通信协议, 本案例功能为配置指定的 CAN 进行输入与输出。

本案例适用的开发板型号:

- HZ-RK3562-SP_EVM
- HZ-RK3562_EVM

1.5.2 操作说明

(4) 赋予可执行权限

将本案例根目录下的可执行文件 `can_demo` 拷贝至开发板文件系统中, 需要确保文件具有可执行权限:

```
chmod +x can_demo
```

```
./can_demo
```

(5) 案例详情

直接运行案例, 不带任何参数, 将会输出案例的使用介绍与详情, 如图 1.5-1。

```
用法 :
./can_demo <设备名> {-r | -w [数据]} [-b 波特率]

参数说明 :
设备名      CAN设备名称, 例如 : can0
选项 :
-w          发送十六进制数据
-r          接收数据
-b          设置波特率 (默认 : 500000)

说明 :
数据必须是十六进制格式, 如 "01 02 03" 或 "010203"
支持空格、逗号分隔或无分隔符的十六进制字符串
每字节必须是两位十六进制数 (00-FF)

示例 :
接收数据 : ./can_demo can0 -r
发送数据 : ./can_demo can0 -w "01 02 03 04"
发送数据 : ./can_demo can0 -w 01020304
指定波特率 : ./can_demo can0 -w "01 02 03 04" -b 250000
```

图 1.5-1 can_demo 详情

(6) 使用示例

```
./can_demo can0 -w "01 02 03 04 05 06 07 08" -b 500000
```

表示通过 CAN0 发送十六进制数据 01 02 03 04 05 06 07 08，配置 CAN 通信波特率为 500k。

```
./can_demo can0 -r -b 500000
```

表示持续监听 CAN0 并接收数据，配置 CAN 通信波特率为 500k。

1.5.3 运行结果

使用 1.5.2 (3) 中的示例：

可参考对应板卡型号的《测试手册》中 CAN 功能测试小节，正确连接并使用 USB-CAN 适配器，根据实际需求选择目标测试引脚。

- CAN 发送数据时，PC 会接收到板卡发送的信息。
- CAN 打开接收模式时，将会收到 PC 发送的信息。

The screenshot displays the execution of the `can_demo` application in a terminal window and the `USB-CAN Tool` interface. The terminal shows the command `./can_demo can0 -w "01 02 03 04 05 06 07 08" -b 500000` being executed, followed by successful transmission of the data sequence `01 02 03 04 05 06 07 08`. A red arrow points from the terminal output to the `USB-CAN Tool` interface, which shows the received data in the `统计数据: 通道1` (Statistics: Channel 1) section. The `USB-CAN Tool` interface also shows the `发送数据` (Send Data) section with the same data sequence. The `统计数据: 通道2` (Statistics: Channel 2) section shows the received data sequence `08 07 06 05 04 03 02 01`. The `统计数据: 通道1` section contains a table of received data:

序号	系统时间	时间标识	CAN通道	传输方向	ID号	帧类型	帧格式	长度	数据
00000	17:44:40.608	0x42D67E2	ch2	接收	0x0123	数据帧	标准帧	0x08	x 01 02 03 04 05 06 07 08
00001	17:44:41.598	0x42D6EF1	ch2	接收	0x0123	数据帧	标准帧	0x08	x 01 02 03 04 05 06 07 08
00002	17:44:42.588	0x42D6B03	ch2	接收	0x0123	数据帧	标准帧	0x08	x 01 02 03 04 05 06 07 08
00003	17:44:43.608	0x42D6D14	ch2	接收	0x0123	数据帧	标准帧	0x08	x 01 02 03 04 05 06 07 08
00004	17:44:44.698	0x42D6425	ch2	接收	0x0123	数据帧	标准帧	0x08	x 01 02 03 04 05 06 07 08
00005	17:45:02.286	无	ch2	发送	0x0000	数据帧	标准帧	0x08	x 08 07 06 05 04 03 02 01
00006	17:45:03.496	无	ch2	发送	0x0000	数据帧	标准帧	0x08	x 08 07 06 05 04 03 02 01
00007	17:45:04.606	无	ch2	发送	0x0000	数据帧	标准帧	0x08	x 08 07 06 05 04 03 02 01
00008	17:45:05.623	无	ch2	发送	0x0000	数据帧	标准帧	0x08	x 08 07 06 05 04 03 02 01
00009	17:45:07.536	无	ch2	发送	0x0000	数据帧	标准帧	0x08	x 08 07 06 05 04 03 02 01

图 1.5-2 can_demo 运行结果

1.5.4 部分代码

1) ReceiveMode - 通过调用 CanRead 函数，接收数据并打印。

```
// 接收模式
void ReceiveMode(CanInfoObj *pCanInfo)
{
    int ret, i;
    char read_buf[MAX_BUF_LEN] = {0};

    printf("接收模式 - 按Ctrl+C退出\n");

    while (1)
    {
        memset(read_buf, 0, sizeof(read_buf));
        ret = CanRead(pCanInfo, read_buf, sizeof(read_buf));

        if (ret < 0)
        {
            printf("接收错误: %d\n", ret);
            sleep(1);
            continue;
        }
        else if (ret > 0)
        {
            printf("接收到数据: ID=0x%X, 长度=%d, 数据=[",
                   pCanInfo->CanRead.canid, pCanInfo->CanRead.len);

            // 只显示十六进制值
            for (i = 0; i < pCanInfo->CanRead.len; i++)
            {
                printf("%02X ", pCanInfo->CanRead.data[i]);
            }
            printf("]\n");

            // 短暂延时, 避免CPU占用过高
            usleep(10000);
        }
    }
}
```

图 1.5-3 can_demo 接收数据并打印

2) SendMode- 调用 ParseHexString 函数，解析输入的字符；调用 CanWrite 函数，将数据通过指定 CAN 端口发送出去：

```
// 发送模式
void SendMode(CanInfoObj *pCanInfo, const char *message)
{
    int ret, i;
    uint8_t send_data[MAX_BUF_LEN] = {0};
    int data_len = 0;

    // 默认数据
    if (message == NULL) ...
    else
    {
        // 解析输入的十六进制字符串
        data_len = ParseHexString(message, send_data, MAX_BUF_LEN);
        if (data_len == 0) ...
    }

    printf("发送模式 - 按Ctrl+C退出\n");
    printf("发送数据: [");
    for (i = 0; i < data_len; i++) ...
    printf("]\n");

    while (1)
    {
        ret = CanWrite(pCanInfo, (char *)send_data, data_len);

        if (ret < 0)
        {
            printf("发送错误: %d\n", ret);
            sleep(1);
            continue;
        }

        printf("发送成功: [");
        for (i = 0; i < data_len; i++)
        {
            printf("%02X ", send_data[i]);
        }
        printf("]\n");

        sleep(1);
    }
}
```

图 1.5-4 can_demo 发送输入的字符